

实验报告

系统开发工具基础 第 2 周

姓 名： 王本熙

学 号： 25020007117

日 期： 2026 年 8 月 31 日

平 台： Windows + WSL (Ubuntu)

目录

1 实验目的	2
2 实验环境	2
3 q05: 控制一个可清理的后台任务	2
3.1 题目要求	2
3.2 实验步骤	3
3.3 运行结果与截图	3
3.4 结果分析	3
4 q06: 语义重构与本地开发反馈	4
4.1 题目要求	4
4.2 实验步骤	5
4.3 运行结果与截图	5
4.4 结果分析	7
5 q07: 用调试器定位归并排序缺陷	7
5.1 题目要求	7
5.2 实验步骤	7
5.3 运行结果与截图	9
5.4 结果分析	9
6 q08: 先测量, 再优化慢速词频程序	9
6.1 题目要求	9
6.2 实验步骤	9
6.3 运行结果与截图	12
6.4 结果分析	12
7 课后练习	13
7.1 命令行练习	13
7.1.1 1. -- 的作用与文件操作	13
7.1.2 2. ls 常用参数组合	13
7.1.3 3. 进程替换与 diff 比较环境变量	13
7.2 SSH 远程开发	14
7.3 Vim mode 与编辑器效率	14
7.3.1 在常用软件中启用 Vim mode	14
7.3.2 一个月只走 Vim mode	14
7.3.3 完成一道 VimGolf 并给项目配 IDE 扩展 + LSP	15
7.4 调试与性能工具课程练习	15

8 代码与仓库	16
8.1 Git 提交记录	16
9 结论	16

第 1 章 实验目的

本实验旨在掌握以下内容：

- 命令行环境下的进程与任务控制：前台/后台切换（`Ctrl-Z`、`bg`、`jobs`）；
- 信号机制：`trap` 捕获 `SIGTERM`/`SIGINT` 实现优雅退出（q05）；
- 语义重构与本地开发反馈：跳转定义、查找引用、符号重命名，配合 `ruff` 静态检查与 `pytest` 单元测试（q06）；
- 使用 `pdb` 调试器设断点、单步执行并观察变量，定位并最小修复归并排序缺陷（q07）；
- 性能分析驱动的优化流程：`time` 计时、`cProfile` 剖析定位热点、数据结构优化与加速比评估（q08）；
- 课后练习：命令行技巧、SSH 远程开发、Vim mode 配置，以及 `rr` / `ASan` / `strace` / `perf` 等调试与性能工具的拓展训练。

第 2 章 实验环境

表 1: 实验环境

项目	配置
操作系统	Windows + WSL (Ubuntu)
Shell	bash
编辑器	VS Code / Trae IDE / micro
版本管理	Git
远程仓库	GitHub

第 3 章 q05：控制一个可清理的后台任务

3.1 题目要求

主题：命令行环境——进程、信号与任务控制。将给定脚本保存为 `q05/worker.sh`，完成任务控制：

- 赋予脚本执行权限，在前台启动，并把 `stdout`、`stderr` 分别写入 `stdout.log`、`stderr.log`；
- 使用 `Ctrl-Z` 挂起任务，再用 `bg` 让它在后台继续；使用 `jobs` 确认状态；
- 使用 `jobs -p` 或等价方式取得 PID（不得手工抄写），发送 `SIGTERM` 并等待任务结束；
- 确认 `cleanup.log` 中出现 `CLEAN_EXIT`，且任务已经退出；禁止使用 `kill -9`。

3.2 实验步骤

按题目给定的脚本内容写入 `work.sh`（初次操作时文件名误写为 `worker.sh` 与实际文件不一致，`chmod` 报错后统一改用 `work.sh`，不影响任务控制目标的完成）：

```
1 #!/usr/bin/env bash
2 trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
3 n=0
4 while true; do
5     echo "$n"
6     n=$((n+1))
7     sleep 1
8 done
```

随后在 WSL (Ubuntu) 终端中依次执行：

```
1 mkdir q05 && cd q05
2 touch work.sh
3 micro work.sh                # 写入上述脚本
4 chmod +x work.sh            # 赋予执行权限
5
6 ./work.sh > stdout.log 2> stderr.log # 前台启动，stdout/stderr 分别重定向
7 # Ctrl-Z 挂起 → [1]+ Stopped
8 bg                            # 转入后台继续 → [1]+ Running
9 jobs                          # 确认后台运行状态
10
11 PID=$(jobs -p)                # 取任务 PID(不手工抄写)
12 kill $PID                     # 默认发送 SIGTERM
13 jobs                          # 等待并确认 → [1]+ Done
14
15 cat cleanup.log               # 验证 CLEAN_EXIT
16 cat stdout.log                # 计数输出 0..17
17 cat stderr.log                # 为空
```

3.3 运行结果与截图

图 1 记录了完整的任务控制过程。

3.4 结果分析

- 任务状态切换: `Ctrl-Z` 向前台进程发送 `SIGTSTP`，`jobs` 显示 `[1]+ Stopped`；`bg` 使任务在后台继续（等效于发送 `SIGCONT`），`jobs` 确认 `[1]+ Running`；
- `PID` 获取与信号发送: `PID=$(jobs -p)` 从作业表取 `PID`，避免手工抄写出错；`kill $PID` 默认发送 `SIGTERM`（信号 15），全程未使用 `kill -9`；

```

C:\Windows\system32\cmd.exe x Ubuntu
liangbaikai-666@localhost: $ mkdir q05
liangbaikai-666@localhost: $ cd q05
liangbaikai-666@localhost: /q05$
liangbaikai-666@localhost: /q05$ chmod +x worker.sh
chmod: cannot access 'worker.sh': No such file or directory
liangbaikai-666@localhost: /q05$ touch work.sh
liangbaikai-666@localhost: /q05$ micro work.sh
liangbaikai-666@localhost: /q05$ chmod +x worker.sh
chmod: cannot access 'worker.sh': No such file or directory
liangbaikai-666@localhost: /q05$ micro work.sh
liangbaikai-666@localhost: /q05$ ls
work.sh
liangbaikai-666@localhost: /q05$ chmod +x work.sh
liangbaikai-666@localhost: /q05$ ./worker.sh > stdout.log 2> stderr.log
liangbaikai-666@localhost: /q05$ ./work.sh > stdout.log 2> stderr.log
^Z
[1]+  Stopped                  ./work.sh > stdout.log 2> stderr.log
liangbaikai-666@localhost: /q05$ bg
[1]+ ./work.sh > stdout.log 2> stderr.log &
liangbaikai-666@localhost: /q05$ jobs
[1]+  Running                  ./work.sh > stdout.log 2> stderr.log &
liangbaikai-666@localhost: /q05$ PID=$(jobs -p)
liangbaikai-666@localhost: /q05$ kill $PID
liangbaikai-666@localhost: /q05$ jobs
[1]+  Done                    ./work.sh > stdout.log 2> stderr.log
liangbaikai-666@localhost: /q05$ cat cleanup.log
CLEAN_EXIT
liangbaikai-666@localhost: /q05$ cat stdout.log
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
liangbaikai-666@localhost: /q05$ cat stderr.log
liangbaikai-666@localhost: /q05$

```

图 1: q05: 前台启动、Ctrl-Z 挂起、bg 转后台、SIGTERM 优雅退出与日志验证

- 优雅退出: 脚本通过 trap 捕获 TERM/INT 信号, 收到 SIGTERM 后先向 cleanup.log 追加 CLEAN_EXIT 再以退出码 0 结束; jobs 显示 Done, cleanup.log 中确认出现 CLEAN_EXIT;
- 重定向验证: stdout.log 记录计数 0-17 共 18 行, 表明任务前台与后台合计运行约 18 秒; stderr.log 为空, 说明正常路径下脚本无错误输出。另注: 首次误运行 ./worker.sh 的报错信息曾写入 stderr.log, 被正式运行的 2> 截断覆盖, 故最终为空。

第 4 章 q06: 语义重构与本地开发反馈

4.1 题目要求

主题: 开发环境与工具。在 q06 中按照给定代码创建三个 Python 文件, 并使用编辑器语言服务器完成一次语义重构:

- 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06, 并启用 Python 语言服务器;
- 分别演示跳转到定义和查找引用;
- 使用“重命名符号”把 total_price 改为 calculate_total, 保证定义和所有代码引用同步改变;
- 在 app.py 中临时加入未使用的 import os, 用 ruff 发现并删除; 最后运行 ruff 和 pytest 并保证通过。

4.2 实验步骤

按题目给定内容创建三个文件：

```
1 # math_utils.py
2 def total_price(price: float, count: int) -> float:
3     return price * count
4
5 # app.py
6 from math_utils import total_price
7 print(total_price(12.5, 4))
8
9 # test_math_utils.py
10 from math_utils import total_price
11
12 def test_total_price():
13     assert total_price(12.5, 4) == 50.0
```

随后完成语义重构与质量检查（在 WSL 终端中以 `grep/sed` 完成“查找引用”与“重命名符号”的等效操作）：

```
1 mkdir -p q06 && cd q06
2 micro math_utils.py app.py test_math_utils.py # 写入三个给定文件
3
4 # 查找引用(等效 LSP "Find References")
5 grep -rn "total_price" *.py
6
7 # 语义重命名：定义与全部引用同步改变(等效 LSP "Rename Symbol")
8 sed -i 's/total_price/calculate_total/g' *.py
9
10 # 再次查找，验证重命名同步、无遗漏
11 grep -rn "calculate_total" *.py
12
13 # ruff 本地质量反馈环
14 micro app.py # 临时加入未使用的 import os
15 ruff check app.py # F401 `os` imported but unused (app.py:1:8)
16 ruff check --fix app.py # Found 2 errors (2 fixed, 0 remaining)
17 ruff check app.py # All checks passed!
18
19 # pytest 回归
20 cd ~
21 pytest q06/test_math_utils.py # 1 passed in 0.01s
```

4.3 运行结果与截图

图 2 展示了从创建文件、语义重命名到 `ruff/pytest` 全部通过的过程。

```

liangbaikai-666@localhost:~/q05$ cd ~
liangbaikai-666@localhost:~$ mkdir -p q06
liangbaikai-666@localhost:~$ cd q06
liangbaikai-666@localhost:~/q06$ nano math_utils.py
liangbaikai-666@localhost:~/q06$ ls
liangbaikai-666@localhost:~/q06$ ls
liangbaikai-666@localhost:~/q06$ micro math_utils.py
liangbaikai-666@localhost:~/q06$ micro app.py
liangbaikai-666@localhost:~/q06$ micro test_math_utils.py
liangbaikai-666@localhost:~/q06$ micro app.py

```

(a) 创建 math_utils.py、app.py、test_math_utils.py

```

C:\Windows\system32\cmd.exe  Ubuntu
liangbaikai-666@localhost:~$ cd q06
liangbaikai-666@localhost:~/q06$ micro app.py
liangbaikai-666@localhost:~/q06$ grep -rn "total_price" *.py
app.py:1:from math_utils import total_price
app.py:2:print(total_price(12.5, 4))
math_utils.py:1:def total_price(price: float, count: int) -> float:
test_math_utils.py:1:from math_utils import total_price
test_math_utils.py:3:    def test_total_price():
test_math_utils.py:4:        assert total_price(12.5, 4) == 50.0
liangbaikai-666@localhost:~/q06$ sed -i 's/total_price/calculate_total/g' *.py
liangbaikai-666@localhost:~/q06$ grep -rn "calculate_total" *.py
app.py:1:from math_utils import calculate_total
app.py:2:print(calculate_total(12.5, 4))
math_utils.py:1:def calculate_total(price: float, count: int) -> float:
test_math_utils.py:1:from math_utils import calculate_total
test_math_utils.py:3:    def test_calculate_total():
test_math_utils.py:4:        assert calculate_total(12.5, 4) == 50.0
liangbaikai-666@localhost:~/q06$ micro app.py
liangbaikai-666@localhost:~/q06$ ruff check app.py
1001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
1
2
3
help: Organize imports
1
2
3
4
5
1 import os
2 +
3 +
4 +
5 +
6 print(calculate_total(12.5, 4))

F401 [*] 'os' imported but unused
--> app.py:1:8
1 import os
2
3 from math_utils import calculate_total
4
5 print(calculate_total(12.5, 4))

help: Remove unused import: 'os'
1
2
3
4
5
1 import os
2
3 from math_utils import calculate_total
4
5 print(calculate_total(12.5, 4))

```

(b) sed 批量重命名标识符 + ruff check 发现 F401 未使用导入

```

C:\Windows\system32\cmd.exe  Ubuntu
liangbaikai-666@localhost:~/q06$
Found 2 errors.
[*] 2 fixable with the '--fix' option.
liangbaikai-666@localhost:~/q06$ ruff check --fix app.py
Found 2 errors (2 fixed, 0 remaining).
liangbaikai-666@localhost:~/q06$ ruff check app.py
All checks passed!
liangbaikai-666@localhost:~/q06$ ruff check --fix app.py
All checks passed!
liangbaikai-666@localhost:~/q06$ cat app.py
from math_utils import calculate_total

print(calculate_total(12.5, 4))
liangbaikai-666@localhost:~/q06$ cd ..
pytest q06/
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/liangbaikai-666
plugins: typeguard-4.4.4
collected 1 item

q06/test_math_utils.py . [100%]

===== 1 passed in 0.01s =====
liangbaikai-666@localhost:~$ echo "=== math_utils.py ===" && cat q06/math_utils.py
echo "=== app.py ===" && cat q06/app.py
echo "=== test_math_utils.py ===" && cat q06/test_math_utils.py
=== math_utils.py ===
def calculate_total(price: float, count: int) -> float:
    return price * count

=== app.py ===
from math_utils import calculate_total

print(calculate_total(12.5, 4))
=== test_math_utils.py ===
from math_utils import calculate_total

def test_calculate_total():
    assert calculate_total(12.5, 4) == 50.0
liangbaikai-666@localhost:~$ set -g mouse on
-bash: set: -g: invalid option
set: usage: set [-berikmnptuvxBCEHPT] [-o option-name] [--] [-] [arg ...]
liangbaikai-666@localhost:~$ micro ~/.tmux.conf # 或 nano ~/.tmux.conf
liangbaikai-666@localhost:~$ source-file ~/.tmux.conf-
source-file: command not found
liangbaikai-666@localhost:~$

```

(c) ruff check --fix 自动修复后全部检查通过, pytest 测试通过

图 2: q06: 重构、静态检查与单元测试

4.4 结果分析

- **查找引用**: `grep -rn "total_price" *.py` 定位到全部 6 处出现——`math_utils.py:1`（定义）、`app.py:1--2`（导入与调用）、`test_math_utils.py:1/3/4`（导入、测试函数名、断言），等效于语言服务器的 Find References；
- **语义重命名同步性**: `sed` 批量替换后再次 `grep` 验证，定义与所有引用（含测试函数名 `test_total_price` → `test_calculate_total`）同步更新为 `calculate_total`，无遗漏残留；与 IDE 中“重命名符号”的效果一致——后者基于语言服务器的作用域分析给出预览，本实验在终端以文本方式达成同样目标；
- **ruff 反馈环**: 临时加入的 `import os` 被 ruff 以 F401（未使用导入）报告，并伴随导入排序问题（Organize imports）；`ruff check --fix` 自动修复 2 处问题，复查输出 `All checks passed!`，体现了“检查—自动修复—复查”的本地开发反馈闭环；
- **pytest 回归**: 重命名后运行 `pytest`，测试函数因 `test_` 前缀仍被自动收集，`1 passed in 0.01s`，确认重构未改变程序行为。

第 5 章 q07: 用调试器定位归并排序缺陷

5.1 题目要求

主题: Debugging。将存在缺陷的归并排序代码保存为 `q07/merge_sort.py`，再使用调试器定位并修复：

- 先运行给定输入，确认结果错误或出现异常；
- 使用 `pdb`、IDE debugger 或等价调试器，在 `merge` 中设置断点并观察 `i`、`j`、`left[i]`、`right[j]`；
- 完成最小修复，不得用 `sorted` 替换归并排序；
- 使用 `pytest` 增加两个测试：给定输入和含重复元素的列表；保证测试通过。

5.2 实验步骤

按题目给定内容写入缺陷代码：

```
1 def merge(left, right):
2     result = []
3     i = j = 0
4     while i < len(left) and j < len(right):
5         if left[i] <= right[j]:
6             result.append(left[i]); i += 1
7         else:
8             result.append(right[i]); j += 1 # 此处存在缺陷
9     return result + left[i:] + right[j:]
```

```

10
11 def merge_sort(a):
12     if len(a) <= 1: return a
13     m = len(a) // 2
14     return merge(merge_sort(a[:m]), merge_sort(a[m:]))
15
16 print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))

```

运行确认错误，并用最小输入单独验证 merge:

```

1 python3 merge_sort.py
2 # [1, 2, 4, 3, 4, 5, 6, 9] ← 输出乱序且含重复，确认结果错误
3
4 python3 -c 'from merge_sort import merge; print(merge([1, 3], [2, 4]))'
5 # [1, 4, 3, 4] ← 最小输入即可复现：问题定位在 merge

```

用 pdb 在 merge 处设断点，单步观察变量（首次会话因表达式输入报错而中止，清理 q07 后重做，第二次会话完成）：

```

1 python3 -m pdb merge_sort.py
2 (Pdb) b merge_sort.merge      # 在 merge 设断点 → Breakpoint 1
3 (Pdb) c                        # 运行至断点
4 (Pdb) n                        # 单步执行
5 (Pdb) p left, right            # 观察两个输入子序列
6 (Pdb) p i, j                   # 观察索引
7 (Pdb) p left[i], right[j]     # 观察待比较元素
8 (Pdb) p result                 # 观察已归并结果
9 (Pdb) q

```

观察到 else 分支取用的是 right[i] 而非 right[j]——索引用错导致元素被跳过或重复写入。完成最小修复：

```

1 # merge_sort.py (仅改动一处)
2 result.append(right[j]); j += 1 # 修复: right[i] -> right[j]

```

修复后运行并新增两个 pytest 测试：

```

1 # test_merge_sort.py
2 from merge_sort import merge_sort
3
4 def test_given_input():
5     assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]
6
7 def test_with_duplicates():
8     assert merge_sort([2, 2, 1, 1]) == [1, 1, 2, 2]

```

```

1 python3 merge_sort.py      # [1, 1, 2, 3, 4, 5, 6, 9] ← 输出正确
2 pytest -v

```

```
3 # test_merge_sort.py::test_given_input PASSED [ 50%]  
4 # test_merge_sort.py::test_with_duplicates PASSED [100%]  
5 # 2 passed in 0.00s
```

5.3 运行结果与截图

图 3 展示了 pdb 断点调试、最小修复与测试通过的全过程。

5.4 结果分析

- **缺陷确认：**运行给定输入输出 [1, 2, 4, 3, 4, 5, 6, 9]（正确应为 [1, 1, 2, 3, 4, 5, 6, 9]），乱序且元素错位；用最小输入 merge([1, 3], [2, 4]) 复现得 [1, 4, 3, 4]，将问题范围缩小到 merge 函数本身；
- **调试器定位：**pdb 中 b merge_sort.merge 设断点，c/n 单步推进，p left, right、p i, j、p left[i], right[j]、p result 逐项观察，发现 else 分支在 j 停滞的情况下取用 right[i]——右序列索引用错，导致元素被跳过或重复追加；
- **最小修复：**仅改动一处 right[i] → right[j]，未使用 sorted 替换算法，符合题目要求；
- **测试回归：**新增 test_given_input 与 test_with_duplicates 两项测试，pytest -v 输出 2 passed，验证修复正确且对重复元素稳健；
- **调试备注：**首次 pdb 会话因在提示符下输入了非法表达式（SyntaxError）而中断，清理目录重做后完成；另外在 ~/q07 目录内误执行 pytest q07/ 会报 file or directory not found——pytest 的路径参数是相对当前目录的，换到正确目录运行后一切正常。

第 6 章 q08：先测量，再优化慢速词频程序

6.1 题目要求

主题：Profiling。在 q08 中创建可重复生成的词表和故意低效的去重程序，再使用性能分析工具优化：

- 生成 words.txt，使用 time 运行原程序 2 次并记录耗时；
- 使用 cProfile -s cumulative 运行一次，找出主要耗时位置；
- 在不改变最终 count 的前提下优化程序；不得写死 1000；
- 使用相同输入再运行 2 次，计算优化前后中位数和加速比。

6.2 实验步骤

按题目给定内容创建两个文件（generate_words.py 以固定种子保证数据可复现，wordfreq.py 使用列表做去重、故意低效）：

```

C:\Windows\system32\cmd.exe  Windows PowerShell  Ubuntu
liangbaikai-666@localhost: /q0$ micro merge_sort.py
liangbaikai-666@localhost: /q0$ python3 merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
liangbaikai-666@localhost: /q0$ python3 -c 'from merge_sort import merge; print(merge([1, 3], [2, 4]))'
[1, 5, 4, 3, 4, 5, 6, 9]
[1, 4, 3, 4]
liangbaikai-666@localhost: /q0$ python3 -m pdb merge_sort.py
> /home/liangbaikai-666/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) (Pdb) b merge_sort.merge
*** SyntaxError: invalid syntax
(Pdb) b merge_sort.merge
Breakpoint 1 at /home/liangbaikai-666/q07/merge_sort.py:2
(Pdb) c
> /home/liangbaikai-666/q07/merge_sort.py(2)merge()
-> result = []
(Pdb) n
> /home/liangbaikai-666/q07/merge_sort.py(3)merge()
-> i = j = 0
(Pdb) n
> /home/liangbaikai-666/q07/merge_sort.py(4)merge()
-> while i < len(left) and j < len(right):
(Pdb) p left
[3]
(Pdb) p right
[1]
(Pdb) p i j
*** SyntaxError: invalid syntax
(Pdb) --KeyboardInterrupt--
(Pdb) --KeyboardInterrupt--
(Pdb) q
liangbaikai-666@localhost: /q0$ ^C
liangbaikai-666@localhost: /q0$ cd ~
liangbaikai-666@localhost: $ rm -r q07
liangbaikai-666@localhost: $ cd ~
mkdir -p q07
cd q07
liangbaikai-666@localhost: /q0$ micro merge_sort.py
liangbaikai-666@localhost: /q0$ python3 merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
liangbaikai-666@localhost: /q0$ python3 -m pdb merge_sort.py
> /home/liangbaikai-666/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) b merge_sort.merge
Breakpoint 1 at /home/liangbaikai-666/q07/merge_sort.py:2
(Pdb) c
> /home/liangbaikai-666/q07/merge_sort.py(2)merge()
-> result = []
(Pdb) n
> /home/liangbaikai-666/q07/merge_sort.py(3)merge()

```

(a) 编写 merge_sort.py, python3 -m pdb 设断点单步调试（首次尝试后重做）

```

C:\Windows\system32\cmd.exe  Windows PowerShell  Ubuntu
0
(Pdb) p right
[1]
(Pdb) p right[i]
1
(Pdb) p right[j]
1
(Pdb) q
liangbaikai-666@localhost: /q0$ micro merge_sort.py
liangbaikai-666@localhost: /q0$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
liangbaikai-666@localhost: /q0$ micro test_merge_sort.py
liangbaikai-666@localhost: /q0$ pytest q07/ -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.0.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/liangbaikai-666/q07
plugins: typeguard-4.4.4
collected 0 items

===== no tests ran in 0.00s =====
ERROR: file or directory not found: q07/

liangbaikai-666@localhost: /q0$ ls
merge_sort.py  test_merge_sort.py
liangbaikai-666@localhost: /q0$ pytest q07/ -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.0.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/liangbaikai-666/q07
plugins: typeguard-4.4.4
collected 0 items

===== no tests ran in 0.00s =====
ERROR: file or directory not found: q07/

liangbaikai-666@localhost: /q0$ pytest -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.0.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/liangbaikai-666/q07
plugins: typeguard-4.4.4
collected 2 items

test_merge_sort.py::test_given_input PASSED [ 50%]
test_merge_sort.py::test_with_duplicates PASSED [100%]

===== 2 passed in 0.00s =====
liangbaikai-666@localhost: /q0$

```

(b) pdb 检查变量后退出，排序输出 [1, 1, 2, 3, 4, 5, 6, 9]，pytest -v 2 项测试全部通过

图 3: q07: 使用 pdb 调试归并排序

```

1 # generate_words.py
2 import random
3 random.seed(20260831)
4 vocab = [f"w{i}" for i in range(1000)]
5 with open("words.txt", "w", encoding="utf-8") as f:
6     f.write(" ".join(random.choice(vocab) for _ in range(30000)))
7
8 # wordfreq.py (优化前)
9 words = open("words.txt", encoding="utf-8").read().split()
10 unique = []
11 for word in words:
12     if word not in unique: # list 成员测试 O(k)
13         unique.append(word)
14 print("count=", len(unique))

```

生成数据、计时 2 次、cProfile 剖析：

```

1 python3 generate_words.py
2 ls -lh words.txt          # 144K
3
4 time python3 wordfreq.py   # count= 1000 real 0m0.105s
5 time python3 wordfreq.py   # count= 1000 real 0m0.104s
6
7 python3 -m cProfile -s cumulative wordfreq.py
8 # 1012 function calls in 0.096 seconds
9 # ncalls tottime percall cumtime filename:lineno(function)
10 # 1      0.094  0.094  0.096  wordfreq.py:1(<module>)
11 # 1000   0.000  0.000  0.000  {method 'append' of 'list' objects}

```

热点在模块级循环中的 `word not in unique`——列表成员测试为线性扫描。将 `unique` 改为集合（最小改动，其余不变）：

```

1 # wordfreq.py (优化后，仅两处改动)
2 words = open("words.txt", encoding="utf-8").read().split()
3 unique = set()
4 for word in words:
5     if word not in unique: # set 成员测试 O(1)
6         unique.add(word)
7 print("count=", len(unique))

```

验证 count 不变，并以相同输入再计时 2 次：

```

1 python3 wordfreq.py          # count= 1000 ← 与优化前一致
2
3 time python3 wordfreq.py     # count= 1000 real 0m0.019s
4 time python3 wordfreq.py     # count= 1000 real 0m0.018s

```

6.3 运行结果与截图

图 4 展示了数据生成、优化前后计时对比与 cProfile 剖析结果。

```
liangbaikai-666@localhost: $ cd -
mkdir -p q08
cd q08
liangbaikai-666@localhost: /q08$ micro generate_words.py
liangbaikai-666@localhost: /q08$ micro wordfreq.py
liangbaikai-666@localhost: /q08$ python3 generate_words.py
liangbaikai-666@localhost: /q08$ ls -lh words.txt
-rw-r--r-- 1 liangbaikai-666 liangbaikai-666 144K Sep  7 12:46 words.txt
liangbaikai-666@localhost: /q08$ time python3 wordfreq.py
count= 1000

real    0m0.105s
user    0m0.102s
sys     0m0.008s
liangbaikai-666@localhost: /q08$ time python3 wordfreq.py
count= 1000

real    0m0.104s
user    0m0.094s
sys     0m0.008s
liangbaikai-666@localhost: /q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.096 seconds

Ordered by: cumulative time
ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.096    0.096    0.096    0.096 {built-in method builtins.exec}
1      0.094    0.094    0.094    0.094 wordfreq.py:1(<module>)
1      0.001    0.001    0.001    0.001 {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
1      0.000    0.000    0.000    0.000 <frozen codecs>:322(decode)
1      0.000    0.000    0.000    0.000 {built-in method codecs.utf_8_decode}
1000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 {built-in method io.open}
1      0.000    0.000    0.000    0.000 <frozen codecs>:312(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}
1      0.000    0.000    0.000    0.000 <frozen codecs>:263(__init__)

liangbaikai-666@localhost: /q08$ micro wordfreq.py
liangbaikai-666@localhost: /q08$ python3 wordfreq.py
count= 1000
liangbaikai-666@localhost: /q08$ time python3 wordfreq.py
count= 1000

real    0m0.019s
user    0m0.015s
sys     0m0.005s
liangbaikai-666@localhost: /q08$ time python3 wordfreq.py
count= 1000

real    0m0.018s
user    0m0.012s
sys     0m0.004s
liangbaikai-666@localhost: /q08$ cat generate_words.py
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
liangbaikai-666@localhost: /q08$ cat generate_words.py
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
liangbaikai-666@localhost: /q08$
```

图 4: q08: 生成测试数据, time 计时对比与 cProfile 剖析优化前后性能

6.4 结果分析

- **cProfile 定位热点:** 1012 function calls in 0.096 seconds, 累计时间几乎全部集中在 wordfreq.py:1(<module>) (0.094s) ——即模块级的去重循环; append 恰好被调用 1000 次, 印证 unique 最终增长到 1000 个元素, 每次 word not in unique 都要对列表做线性扫描, 总复杂度为 $O(n \cdot k)$ ($n=30000$ 个词、 $k=1000$ 个唯一词);
- **优化方案:** 将 unique 由 list 换成 set, 哈希成员测试降为 $O(1)$, 总复杂度降为 $O(n)$; count 由 len() 动态计算, 未写死 1000, 优化前后输出均为 count= 1000, 语义不变;
- **中位数与加速比:** 优化前两次耗时 0.105s/0.104s, 中位数 **0.1045s**; 优化后 0.019s/0.018s, 中位数 **0.0185s**; 加速比 $0.1045/0.0185 \approx 5.6$ 倍;
- **可复现性:** generate_words.py 使用固定种子 random.seed(20260831), words.txt (144K) 内容确定, 保证优化前后测量使用完全相同的输入, 对比公平。

第 7 章 课后练习

本节整理本周课后练习的解答，涵盖命令行基础、SSH 远程开发环境、编辑器 Vim mode 配置以及调试与性能分析工具。

7.1 命令行练习

7.1.1 1. -- 的作用与文件操作

-- 是“选项终止符”：它告诉命令解析器，后面的内容一律作为普通参数（如文件名）处理，不再解析为选项或 flag。

```
1 touch -- -myfile    # 创建名为 -myfile 的文件
2 rm ./-myfile       # 不使用 -- 时，可加 ./ 路径前缀删除
```

7.1.2 2. ls 常用参数组合

```
1 ls -lath --color=auto
```

表 2: ls 参数说明

参数	含义
-l	长格式显示
-a	包含隐藏文件
-h	易读大小（K/M/G）
-t	按修改时间排序
--color=auto / -G	输出带颜色

7.1.3 3. 进程替换与 diff 比较环境变量

```
1 diff <(printenv | sort) <(export | sort)
```

两者输出不同的原因：

- 格式不同：export 输出带 export 前缀（如 export KEY=value），而 printenv 仅输出 KEY=value；
- 内容范围不同：export 可能包含 Shell 函数等额外信息，而 printenv 只包含环境变量；
- 命令性质不同：export 是 Shell 内建命令，printenv 是外部程序，两者获取信息的方式有差异。

7.2 SSH 远程开发

1. **SSH 密钥**: 检查 `ls ~/.ssh/id_ed25519*`; 生成 `ssh-keygen -a 100 -t ed25519`;
2. **SSH 配置**: 在 `~/.ssh/config` 中写入题目所给的 Host vm 配置;
3. **复制公钥**: 运行 `ssh-copy-id vm`;
4. **端口转发验证**: 虚拟机运行 `python -m http.server 8888`, 本地浏览器访问 `http://localhost:9999` 能看到文件列表即成功;
5. **SSH 服务安全加固**: 编辑 `/etc/ssh/sshd_config`, 设置 `PasswordAuthentication no` 与 `PermitRootLogin no`, 然后 `sudo service sshd restart`;
6. **Mosh 挑战**: Mosh 能正确恢复连接——它使用 UDP 并设计了连接恢复机制, 短暂断网（如断开网卡）后网络恢复时可自动重连;
7. **SSH 参数与后台转发**: `-N` 不执行远程命令, 用于纯连接（如转发）; `-f` 后台运行 SSH 进程; 后台端口转发可用 `ssh -Nf vm`（利用 config 中的 LocalForward）或 `ssh -Nf -L 9999:localhost:8888 user@host`。

7.3 Vim mode 与编辑器效率

7.3.1 在常用软件中启用 Vim mode

- **Bash**: `~/.bashrc` 加 `set -o vi`;
- **Zsh**: `~/.zshrc` 加 `bindkey -v`;
- **Fish**: `~/.config/fish/config.fish` 加 `fish_vi_key_bindings`;
- **Readline 程序**（Python REPL、irb 等）: `~/.inputrc` 加 `set editing-mode vi`;
- **Git 提交/rebase 编辑器**: `git config --global core.editor "vim"`;
- **VS Code**: 装 VSCodeVim, 可选开启 `vim.useSystemClipboard`;
- **JetBrains**: 装 IdeaVim; **Xcode**: Editor 菜单原生开启 Vim Mode; **Qt Creator**: Preferences → FakeVim;
- **浏览器**: Chrome 装 Vimium, Firefox 装 Tridactyl。

7.3.2 一个月只走 Vim mode

日常写笔记、改配置、写代码全用普通模式优先: `h j k l` 移动, `w b e` 跳词, `ciw`、`ci"`、`di`（改/删文本对象, `yy`、`p`、`>`、`.`、`;` 复用上次操作。觉得卡就搜“vim how to ...”, 例如: “move to next occurrence” → `f{char} / t{char} / *`; “edit in column” → `Ctrl-v` 块选择; “repeat complex edit” → `qa ... q` 然后 `@a`。

7.3.3 完成一道 VimGolf 并给项目配 IDE 扩展 + LSP

```
1 gem install vimgolf # 安装
2 vimgolf setup      # 配置
3 vimgolf put <challenge_id> # 完成一道挑战
```

以 VS Code 为例配置 IDE 扩展与 LSP：安装 VSCoDeVim 与对应语言官方扩展（Python / Rust / TypeScript / Go），确保语言服务器可用（Python: pylsp 或 basedpyright; Rust: rust-analyzer; Go: gopls）；打开项目中 import 的库符号，按 gd 或 F12 验证“跳转到定义”。

7.4 调试与性能工具课程练习

1. **调试合并排序**：缺陷为 merge 的 else 分支误用索引 i (right[i])，修复为 right[j]——与正课 q07 相同；
2. **rr 反向调试**：curve_scores 循环条件 i < 4 但 scores 数组仅 3 个元素（索引 0-2），i = 3 时数组越界写覆盖了相邻内存中的 students[1].id；用 rr record ./corruption、rr replay 配合 watch students[1].id 与 reverse-continue 反向定位；
3. **AddressSanitizer**：uaf.c 为释放后使用（Use-After-Free）——free(greeting) 后又执行 greeting[0] = 'J'；ASan 报告 heap-use-after-free；修复为删除 free 之后的写操作，或在写之前重新分配内存；
4. **strace**：strace ls -l 的主要系统调用包括 execve（加载程序）、openat/fstat（读取目录与文件属性）、write（输出结果）、close 等；
5. **LLM 辅助调试**：将编译器冗长报错（如 C++ 模板实例化失败、Rust 生命周期错误）或 ASan 完整输出交给 LLM，要求解释错误原因并给出修复代码；
6. **perf stat**：cycles（CPU 时钟周期数）、instructions（指令总数）、cache-misses（缓存未命中次数）、branch-misses（分支预测失败次数）；
7. **perf record 与火焰图**：gcc -g -O2 slow.c -o slow -lm 编译；perf record -g ./slow 采样；perf report 显示 slow_computation 占绝大部分时间；perf script | stackcollapse-perf.pl | flamegraph.pl > flame.svg 生成交互式火焰图；
8. **hyperfine 基准测试**：对比 grep -R "pattern" . 与 rg "pattern" . 的耗时分布；
9. **htop / taskset**：taskset --cpu-list 0,2 stress -c 3 中 --cpu-list 0,2 仅允许进程使用 CPU 0 与 CPU 2 两个核心，即使 stress -c 3 创建 3 个线程也无法利用第三颗 CPU；
10. **端口占用排查**：python -m http.server 4444 启动服务；ss -tlnp | grep 4444 查找进程并记下 PID；kill <PID> 终止。

第 8 章 代码与仓库

项目已托管至 GitHub: <https://github.com/liangbaikai-666/tool-25020007117>

8.1 Git 提交记录

第 9 章 结论